
PART I — FOUNDATIONS

Chapter 3

Setting Up Your Development Environment

“Give me six hours to chop down a tree and I will spend the first four sharpening the axe.”

Chapter 3 — Setting Up Your Development Environment

A solid foundation determines the ceiling of everything built on top of it. Before any vector is embedded or any document ingested, your machine must be correctly provisioned with the right Python runtime, the right package manager, the right libraries — and the right habits for keeping secrets secret. This chapter walks through every step of that provisioning, from checking your hardware to verifying that every library in the stack imports without error. By the time you reach Section 3.8, your environment will be indistinguishable from the one used to write this book.

There are no shortcuts here, and that is intentional. Environment failures are the leading cause of abandoned projects, not algorithmic complexity. Every subsection that follows was shaped by real errors encountered during the development of this system — the kind of error that produces a cryptic traceback at 11 p.m. and costs two hours of debugging. You will encounter none of them if you follow this chapter in order.

3.1 Checking if Your Laptop Is RAG-Ready

RAG systems are not computationally trivial. They involve loading embedding models into memory, performing matrix multiplications over hundreds or thousands of vectors, persisting data to disk, and routing network requests to LLM APIs. Before installing a single package, it is worth ensuring that your machine meets the minimum requirements — not to gatekeep, but to set accurate expectations about what will and will not be fast.

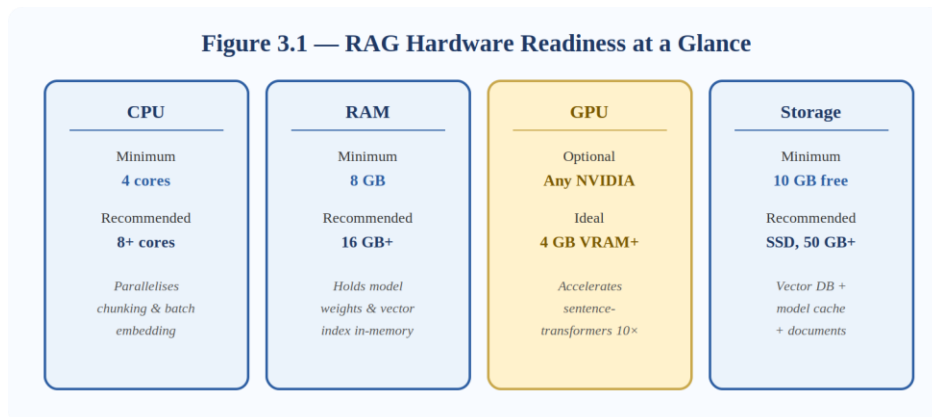


Figure 3.1 — RAG Hardware Readiness at a Glance. The GPU column is optional but accelerates embedding generation by roughly an order of magnitude.

CPU

The CPU handles everything that does not run on a GPU: document parsing, text splitting, ChromaDB queries, LangGraph orchestration, and all Python glue code. A quad-core processor from the last five years is sufficient for development. Eight or more cores will meaningfully reduce batch embedding times when you ingest large document collections.

RAM

RAM is the primary bottleneck for most developer laptops. The sentence-transformers model (all-MiniLM-L6-v2) holds approximately 80 MB of weights in memory, but the surrounding Python runtime, ChromaDB index, and LangGraph state will together consume 2–4 GB during active inference. Eight gigabytes is the practical minimum; sixteen gigabytes allows comfortable multi-tasking. If your machine has only 8 GB and you notice slowdowns, close your browser and IDE before running ingestion.

GPU (Optional but Recommended)

No GPU is required to follow this book. All embedding and inference can run on CPU. However, if you have an NVIDIA GPU with 4 GB or more of VRAM, installing the CUDA build of PyTorch will accelerate sentence-transformers by roughly 10×, reducing batch embedding time from minutes to seconds. The requirements.txt in this project pins torch==2.11.0+cu128 for CUDA 12.8; CPU users should substitute the standard torch wheel.

Storage

SSD storage is strongly preferred over HDD. ChromaDB writes its vector index as a collection of files on disk; frequent small random writes benefit disproportionately from SSD throughput. Reserve at least 10 GB of free space: roughly 5 GB for the Python environment and model cache, and the remainder for your document corpus and vector store. Larger document collections will naturally require more.

A Quick Readiness Check

Run the following in a terminal to surface your system's key specs in one pass:

```
# macOS / Linux
python3 -c "import platform; print(platform.processor())"
python3 -c "import psutil; print(f'RAM: {psutil.virtual_memory().total / 1e9:.1f} GB')"
```

```
# Windows (PowerShell)
wmic cpu get name
systeminfo | findstr /C:"Total Physical Memory"
```

```
# GPU check (requires NVIDIA driver)
nvidia-smi
```

3.2 Python Versions and Why 3.10–3.12 Is the Sweet Spot

Python's ecosystem moves fast, and the ML sub-ecosystem moves even faster. Every major library in this stack — LangChain, ChromaDB, sentence-transformers, LangGraph — targets a specific range of Python interpreter versions, and those ranges do not always overlap neatly with the very latest release.

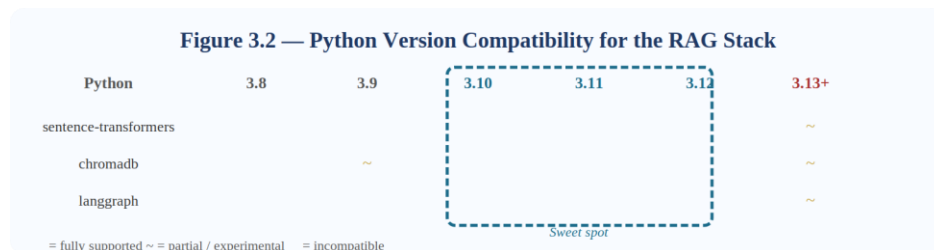


Figure 3.2 — Python Version Compatibility for the RAG Stack. Python 3.10, 3.11, and 3.12 are fully supported by all core libraries. Python 3.13 and newer are experimental at the time of writing.

Definition — Interpreter version pinning

The practice of specifying an exact or bounded Python version for a project, ensuring that the interpreter ABI (Application Binary Interface) matches the compiled extensions in your dependencies. Tools like `uv` and `pyenv` enforce this at environment creation time.

Python 3.10 introduced structural pattern matching (`match / case`), which LangGraph uses internally. Python 3.11 delivered a substantial performance improvement — CPython’s adaptive interpreter reduced execution overhead by 10–60% on typical workloads — meaning the same embedding loop runs measurably faster. Python 3.12 refined that further and stabilised the new `tomllib` standard library module that many modern build tools depend on.

Python 3.13 and 3.14 are too new. Several compiled extensions — notably PyTorch and certain ChromaDB dependencies — do not yet publish wheels for these versions on PyPI, which means `pip` or `uv` fall back to building from source. Source builds require system compilers, C headers, and a significant amount of patience. Unless you specifically enjoy debugging compiler flags, stay within 3.10–3.12.

Analogy — The oven temperature rule

Think of Python versions the way a baker thinks about oven temperatures. The recipe works perfectly at 175°C. You can try 220°C if you like, but you will spend more time watching for smoke than actually baking. Python 3.11 is 175°C.

Installing the Right Version

```
# Check your current version
python3 --version

# Install 3.11 via pyenv (macOS/Linux)
pyenv install 3.11.9
pyenv local 3.11.9

# Install 3.11 on Ubuntu/Debian
sudo add-apt-repository ppa:deadsnakes/ppa
sudo apt install python3.11 python3.11-venv python3.11-dev

# Windows: download the installer from python.org/downloads
# Then verify:
py -3.11 --version
```

3.3 Virtual Environments: `venv` vs `uv` vs `conda`

A virtual environment is an isolated Python installation scoped to a single project. It prevents the single most common source of developer frustration: “*it works on my machine*” failures caused by mismatched package versions between projects. Every serious Python project uses one. The question is which tool creates and manages it.

Definition — Virtual environment

A self-contained directory tree containing a Python interpreter copy and an isolated site-packages directory. Packages installed inside the environment do not affect the system-wide Python installation or any other environment on the machine.

Attribute	venv (stdlib)	uv	conda
Install speed	Moderate (pip)	Very fast (Rust)	Slow (solver)
Non-Python pkgs	No	No	Yes (CUDA, etc.)
Built into Python	Yes	Separate install	Separate install
Best for	Simple projects	This book	Data science

uv is recommended throughout this book for its speed and lock-file reproducibility

Figure 3.3 — Virtual Environment Tool Comparison. The three mainstream choices differ primarily in speed, capability, and setup complexity. *uv* is recommended throughout this book.

venv — The Standard Library Option

Every Python 3.3+ installation includes `venv` in the standard library, requiring no additional installation. It is the safest choice when you cannot install third-party tools and the most widely understood option for onboarding contributors.

```
# Create and activate a venv environment
python3.11 -m venv .venv

# macOS / Linux
source .venv/bin/activate

# Windows (PowerShell)
.\.venv\Scripts\Activate.ps1

# Install packages
pip install -r requirements.txt
```

uv — The Fast Modern Alternative

`uv` is a Rust-based package manager and virtual environment tool. Its dependency resolver runs orders of magnitude faster than `pip` and produces a lock file (`uv.lock`) that pins every transitive dependency to an exact version, guaranteeing reproducibility across machines. This book’s `requirements.txt` was generated using `uv`.

```
# Install uv (once, system-wide)
pip install uv

# Create environment and install all dependencies
uv venv --python 3.11
source .venv/bin/activate          # or .\.venv\Scripts\Activate.ps1 on Windows
uv pip install -r requirements.txt

# For GPU PyTorch, add the PyTorch index:
uv pip install -r requirements.txt --index-url
https://download.pytorch.org/whl/cu128
```

conda — The Data Science Ecosystem

conda manages both Python packages and non-Python system-level dependencies (CUDA toolkits, MKL, HDF5 libraries). It is the preferred tool when your workflow requires binary packages not available on PyPI. Its trade-off is speed: the conda dependency solver is notoriously slow on large environments, and its environments consume significantly more disk space.

For this book, **uv is recommended**. It is faster than pip, produces a reproducible lock file, and handles the PyTorch CUDA index correctly with a single flag. If you already have a conda workflow you are comfortable with, it will work — just ensure your environment uses Python 3.11.

3.4 Installing the Core Stack

The Self-Learning Agentic RAG system rests on six foundational libraries. Each was chosen for a specific reason, and understanding those reasons will serve you well when you need to debug or extend the system.

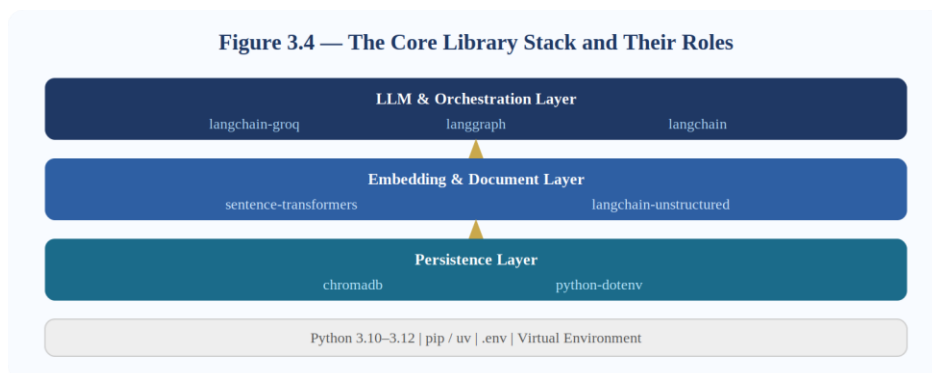


Figure 3.4 — The Core Library Stack and Their Roles. Each layer depends on the one below it; the persistence layer is where data outlives the Python process.

langchain

LangChain provides the abstraction layer that ties together loaders, splitters, retrievers, and LLM calls. Rather than writing bespoke glue code for each component, LangChain exposes a uniform interface — the `Runnable` protocol — through which any component can be composed with any other using the pipe operator (`|`). This composability is what makes swapping out, say, an OpenAI embedder for a local sentence-transformer a one-line change rather than a rewrite.

langchain-groq

The Groq integration gives LangChain access to Llama 3.3 70B running on Groq’s LPU (Language Processing Unit) hardware. Groq’s inference speeds — typically 300–500 tokens per second — are an order of magnitude faster than typical GPU-hosted endpoints, which makes the agentic loop in Chapter 10 feel genuinely interactive rather than sluggish. A free API key is available at console.groq.com.

langchain-unstructured

Document loading is more complex than it appears. PDFs come in dozens of flavours — scanned images, digital text, mixed, form-based — and DOCX files embed styles, tables, and tracked changes. `langchain-unstructured` wraps the Unstructured library, which handles this complexity by detecting document type, extracting text, and returning clean `Document` objects with rich metadata. Chapter 4 goes deep on this component.

langgraph

LangGraph extends LangChain with a graph-based agent runtime. Where LangChain chains execute sequentially, LangGraph allows cycles, conditional branches, and persistent state across steps. This is what

makes the self-learning loop in Part IV possible: the agent can query, evaluate its own output, and decide whether to re-query with a different strategy — all within a single graph execution.

chromadb

ChromaDB is the vector store that persists your document embeddings to disk and serves similarity queries. It runs entirely in-process (no separate server required), persists its index to a local directory, and exposes a simple Python API. It is the right choice for a book project and for small-to-medium production deployments. Chapter 7 covers its internals.

sentence-transformers

sentence-transformers is the library that converts text into dense vector embeddings. The default model used throughout this book is `all-MiniLM-L6-v2`: a 22 MB model that produces 384-dimensional embeddings in milliseconds per sentence. It is fast enough to embed large corpora on a laptop CPU, yet semantically powerful enough to produce high-quality retrieval results.

Installation

```
# With uv (recommended)
uv pip install langchain langchain-groq langchain-unstructured \
               langgraph chromadb sentence-transformers

# With pip
pip install langchain langchain-groq langchain-unstructured \
           langgraph chromadb sentence-transformers
```

3.5 Pinned Versions and Why They Matter

Installing `pip install langchain` today and repeating the same command three months from now will likely install different code. Package maintainers release updates continuously, and a single breaking change in any transitive dependency can silently alter the behaviour of your entire system. The `requirements.txt` file solves this by recording the exact version of every package installed — not just the top-level dependencies, but every package they depend on.

Definition — Pinned dependency

A package specification that fixes an exact version using the `==` operator (e.g., `langchain==1.2.15`), ensuring that anyone who installs from this file gets byte-for-byte identical package code, regardless of when or where the installation occurs.

A tour of the key pinned versions in this project’s requirements.txt:

Package	Pinned Version	Why Pinned
langchain	1.2.15	Major API changes between minor versions; Runnable interface stabilised here
langchain-groq	1.1.2	Groq authentication header format changed in earlier releases
chromadb	1.5.8	Collection API signature changed; older versions incompatible with newer clients
sentence-transformers	5.4.1	Model download URL format changed; embedding output shape must remain consistent
langgraph	1.1.8	State schema and graph compilation API are frequently revised in new releases
torch	2.11.0+cu128	CUDA build pinned to specific toolkit version; CPU/GPU mismatch breaks inference

Table 3.1 — Key pinned versions in this project’s requirements.txt and the reason each matters.

To generate your own requirements.txt after installing packages interactively, run `uv pip freeze > requirements.txt` (or `pip freeze > requirements.txt`). Always commit this file to version control. It is the single most important reproducibility artefact in any Python project.

Common pitfall — Installing packages globally

Never install packages globally when working on this project. A global install can silently override a version in your virtual environment, producing behaviour that is impossible to reproduce. Always confirm that your virtual environment is active (look for `(.venv)` at the start of your terminal prompt) before running any pip or uv command.

3.6 Managing API Keys with .env and python-dotenv

This project requires a Groq API key and, optionally, a LangSmith tracing key. These are secrets — strings that grant authenticated access to paid services. Hardcoding them in source files is a critical security mistake: version control systems persist history indefinitely, and a key committed even once and later deleted remains visible in the repository’s commit log. The standard solution is the .env file pattern, enforced by python-dotenv.

Definition — .env file

A plain-text file, conventionally named `.env`, that contains environment variable assignments in `KEY=VALUE` format. It is loaded at runtime by python-dotenv and is never committed to version control. A `.env.example` file containing placeholder values (but not real keys) is committed instead, serving as documentation.

Creating the .env File

```
# .env (never commit this file)
GROQ_API_KEY=gsk_XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX
LANGCHAIN_API_KEY=ls_XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX
LANGCHAIN_TRACING_V2=true
LANGCHAIN_PROJECT=self-learning-rag
```

Adding .env to .gitignore

```
# .gitignore
.env
.venv/
__pycache__/
*.pyc
data/chroma_db/
```

Loading Keys in Code

```
import os
from dotenv import load_dotenv

# Call once at the entry point of your application
load_dotenv()  # reads .env from the current directory

# Access keys via os.getenv — returns None if not set
groq_key = os.getenv("GROQ_API_KEY")
if not groq_key:
    raise EnvironmentError("GROQ_API_KEY is not set. Check your .env
file.")
```

LangChain and LangGraph read `GROQ_API_KEY` automatically from the environment when you instantiate `ChatGroq`, so you do not need to pass it explicitly to every object. Calling `load_dotenv()` at startup is sufficient.

Analogy — The keychain in your pocket

Think of .env as a keychain that lives in your pocket, not on a public notice board. Your application can reach in and grab a key whenever it needs one. Your collaborators get an identical-looking (but empty) keychain — the .env.example file — so they know which keys to add, but never see your actual values.

3.7 Common Install Errors and How to Fix Them

This section documents the most common installation failures encountered during development of this system. Each error has a root cause and a reliable fix. Read the error message before reaching for a search engine — the answer is usually closer than it appears.

Error 1 — PyTorch Installing CPU-Only When a GPU Is Present

```
# Symptom
import torch
print(torch.cuda.is_available())  # Returns False despite having a GPU

# Root cause: pip installed the CPU-only torch wheel from PyPI

# Fix: reinstall from the PyTorch CUDA index
pip uninstall torch torchvision torchaudio -y
pip install torch torchvision torchaudio \
    --index-url https://download.pytorch.org/whl/cu128
```

```
# Verify
python -c "import torch; print(torch.version.cuda,
torch.cuda.is_available())"
```

Error 2 — libmagic Not Found

```
# Symptom
ImportError: failed to find libmagic. Check your installation

# Fix on Ubuntu / Debian
sudo apt-get install libmagic1

# Fix on macOS
brew install libmagic

# Fix on Windows (ships the DLL inside the wheel)
pip install python-magic-bin
```

The libmagic C library is a system-level dependency that `python-magic` (used by Unstructured) wraps. On Linux and macOS it must be installed via the system package manager. On Windows, `python-magic-bin` ships the DLL alongside the Python wheel, requiring no separate system install.

Error 3 — Windows .venv Locks (Access Denied on Deletion)

```
# Symptom
Remove-Item : Access to the path '.venv\Scripts\...pyd' is denied.

# Root cause: Python has loaded compiled extensions (.pyd files) into
memory.
# Windows locks open files; they cannot be deleted while Python is
running.

# Fix
# 1. Close VS Code, Jupyter, and all terminals
# 2. Optional but reliable: restart the PC
# 3. Force-kill any lingering Python processes:
taskkill /F /IM python.exe

# 4. Delete the locked directory
Remove-Item -Recurse -Force .venv

# 5. Recreate cleanly
uv venv --python 3.11
.\.venv\Scripts\Activate.ps1
uv pip install -r requirements.txt
```

Error 4 — sentence-transformers Fails to Download the Model

```
# Symptom
OSError: [Errno -2] Name or service not known
# or
requests.exceptions.ConnectionError: ...

# Root cause: HuggingFace Hub is blocked on the network, or HF_HOME is not
set.
```

```
# Fix 1: pre-download the model manually on a machine with internet access
from sentence_transformers import SentenceTransformer
SentenceTransformer("all-MiniLM-L6-v2") # downloads and caches

# Fix 2: point HF_HOME to a writable directory
export HF_HOME=/path/to/cache # Linux/macOS
set HF_HOME=C:\cache # Windows cmd
```

Error 5 — LangChain Import Errors After Upgrading

```
# Symptom
ImportError: cannot import name 'RunnablePassthrough' from 'langchain'

# Root cause: LangChain reorganised its module layout in 0.2+.
# Many symbols moved from langchain to langchain_core.

# Fix: update the import path
from langchain_core.runnables import RunnablePassthrough # correct
# NOT: from langchain.schema.runnable import RunnablePassthrough (old)

# If the error persists, check that all langchain-* packages are aligned:
pip show langchain langchain-core langchain-community | grep Version
```

3.8 Verifying the Install — A 5-Minute Smoke Test

A smoke test is a minimal set of checks designed to confirm that the critical path through your system works end-to-end. It does not test correctness in depth — that is the job of a full test suite — but it will surface any broken import, missing credential, or misconfigured environment variable within minutes. Run this test every time you create a new environment or after a major dependency upgrade.

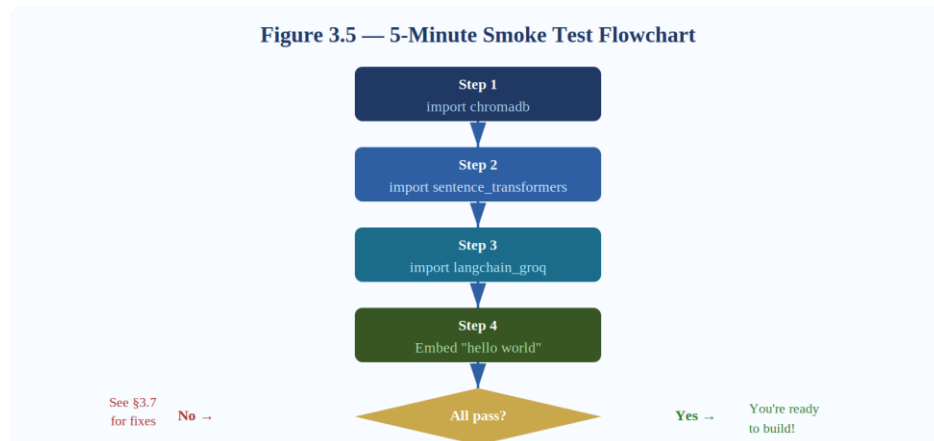


Figure 3.5 — 5-Minute Smoke Test Flowchart. Each step exercises one layer of the stack. A failure at any step points directly to the layer that needs attention.

Smoke Test Script

```
#!/usr/bin/env python3
"""smoke_test.py — run this after setting up the environment."""

import sys
print(f"Python: {sys.version}")

# Step 1: Vector store
import chromadb
```

```

client = chromadb.Client()
col = client.create_collection("smoke_test")
col.add(documents=["hello world"], ids=["1"])
result = col.query(query_texts=["hello"], n_results=1)
assert result["documents"][0][0] == "hello world"
print("✓ chromadb")

# Step 2: Embeddings
from sentence_transformers import SentenceTransformer
model = SentenceTransformer("all-MiniLM-L6-v2")
vec = model.encode("smoke test")
assert vec.shape == (384,)
print(f"✓ sentence-transformers [dim={vec.shape[0]}]")

# Step 3: LangChain + Groq
import os
from dotenv import load_dotenv
load_dotenv()
from langchain_groq import ChatGroq
llm = ChatGroq(model="llama-3.3-70b-versatile", max_tokens=64)
response = llm.invoke("Reply with exactly: SMOKE TEST PASSED")
print(f"✓ langchain-groq [{response.content.strip()[:40]}]")

# Step 4: LangGraph
from langgraph.graph import StateGraph, END
from typing import TypedDict
class State(TypedDict):
    value: int
g = StateGraph(State)
g.add_node("inc", lambda s: {"value": s["value"] + 1})
g.set_entry_point("inc")
g.add_edge("inc", END)
app = g.compile()
out = app.invoke({"value": 0})
assert out["value"] == 1
print("✓ langgraph")

# Step 5: Unstructured loader
from langchain_unstructured import UnstructuredLoader
print("✓ langchain-unstructured")

print("\n All checks passed. Your environment is ready.")

```

Interpreting the Output

A passing run prints a ✓ for each step and ends with “*All checks passed.*” A failure will raise an exception that identifies exactly which library or credential is broken, and the corresponding subsection of this chapter provides the fix.

```

# Run the smoke test
python smoke_test.py

# Expected output
Python: 3.11.9 (main, ...)
✓ chromadb
✓ sentence-transformers [dim=384]
✓ langchain-groq [SMOKE TEST PASSED]

```

```
✓ langgraph
✓ langchain-unstructured
```

```
All checks passed. Your environment is ready.
```

Once this output appears, the environment is verified. You will not need to return to this chapter. The next chapter begins the first substantive topic: the `Document` object — the fundamental unit of data that flows through every stage of the RAG pipeline.